Chapter 4 Threads and Concurrency

Outline

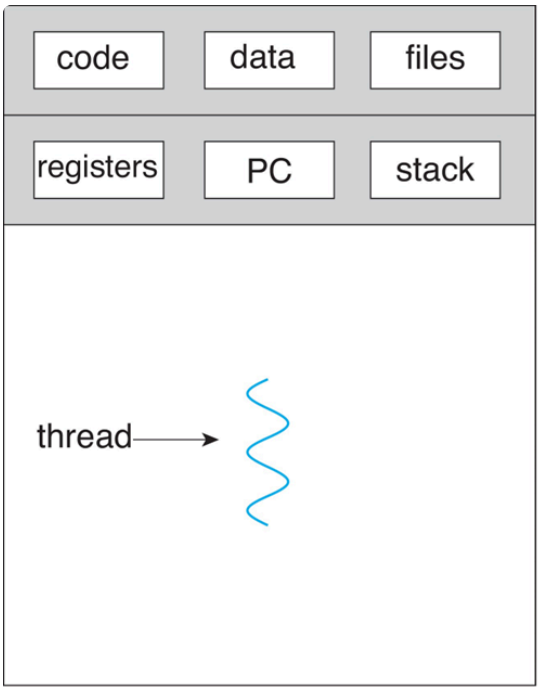
- Overview
- Multicore Programming
- Multithreading Models
- Threads Libraries
- Implicit Threading
- Threading Issues
- Operating Systems Examples

Objectives

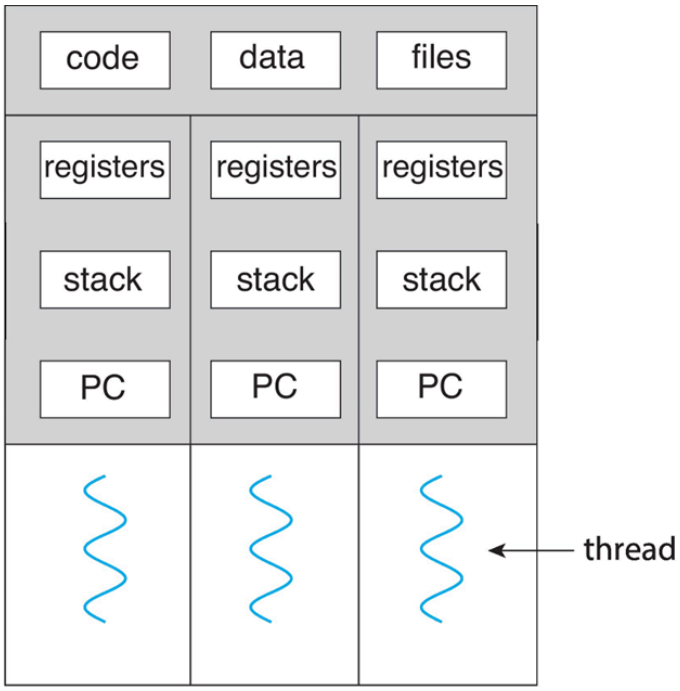
- Identify the basic components of a thread, and contrast threads and processes.
- Describe the benefits and challenges of designing multithreaded applications.
- Illustrate different approaches to implicit threading including thread pools, fork-join and Grand Central Dispatch.
- Describe how the Windows and Linux operating systems represent threads
- Designing multithreaded applications using the Pthreads, Java and Windows threading APIs

Motivation

Most modern applications and kernels are multithreaded. Threads run within applications. Multiple tasks within the application can be implemented by separate threads.
Process creation is heavy-weight while thread creation is light-weight. It can simplify code and increases efficiency.



single-threaded process



multithreaded process

Benefits

- Responsiveness
  Continue execution if part of process is blocked(especially important for user interfaces)
- Resource Sharing
  Threads share resources of process, easier than shared memory or message passing
- Economy
  Cheaper than process creation, thread switching lower overhead than context switching
- Scalability
  Process can take advantage of multicore architectures

Multicore Programming

The challenges of multicore or multiprocessor include:

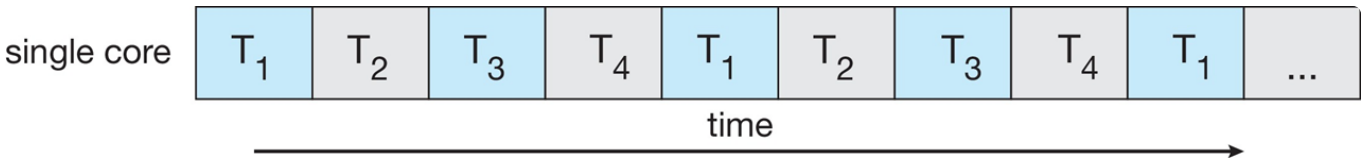
- Dividing activities
- Balance
- Data splitting
- Data dependency
- Testing and debugging

Concurrency vs. Parallelism

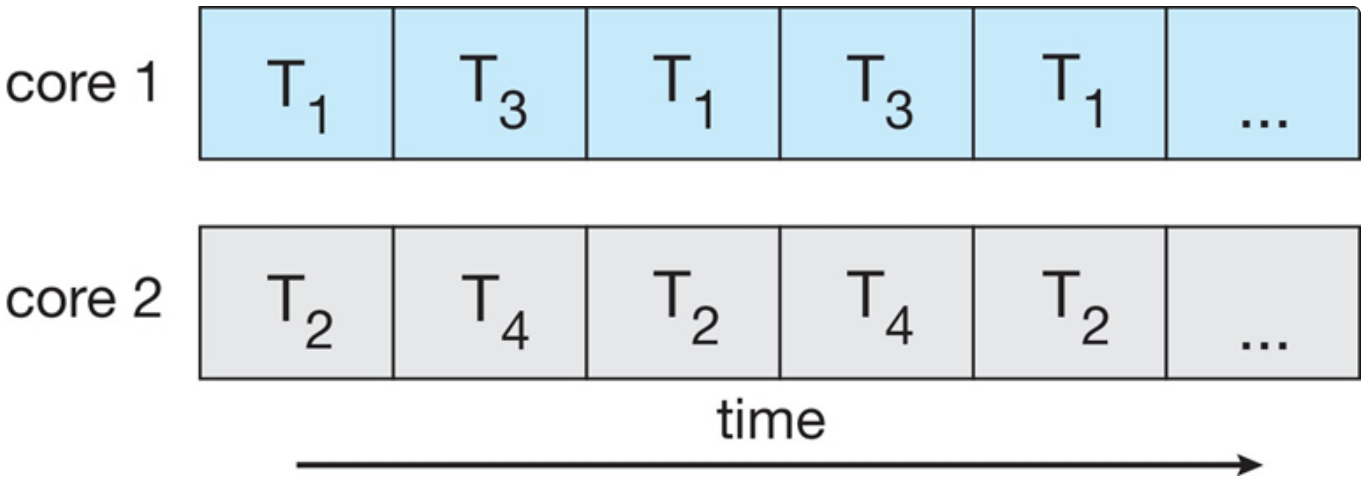
- Parallelism implies a system can perform more than one task simultaneously.(must multi-core system) It is about performance.

- Concurrency supports more than one task making progress.(if single processor, scheduler providing concurrency) It is about architecture.

Concurrency:

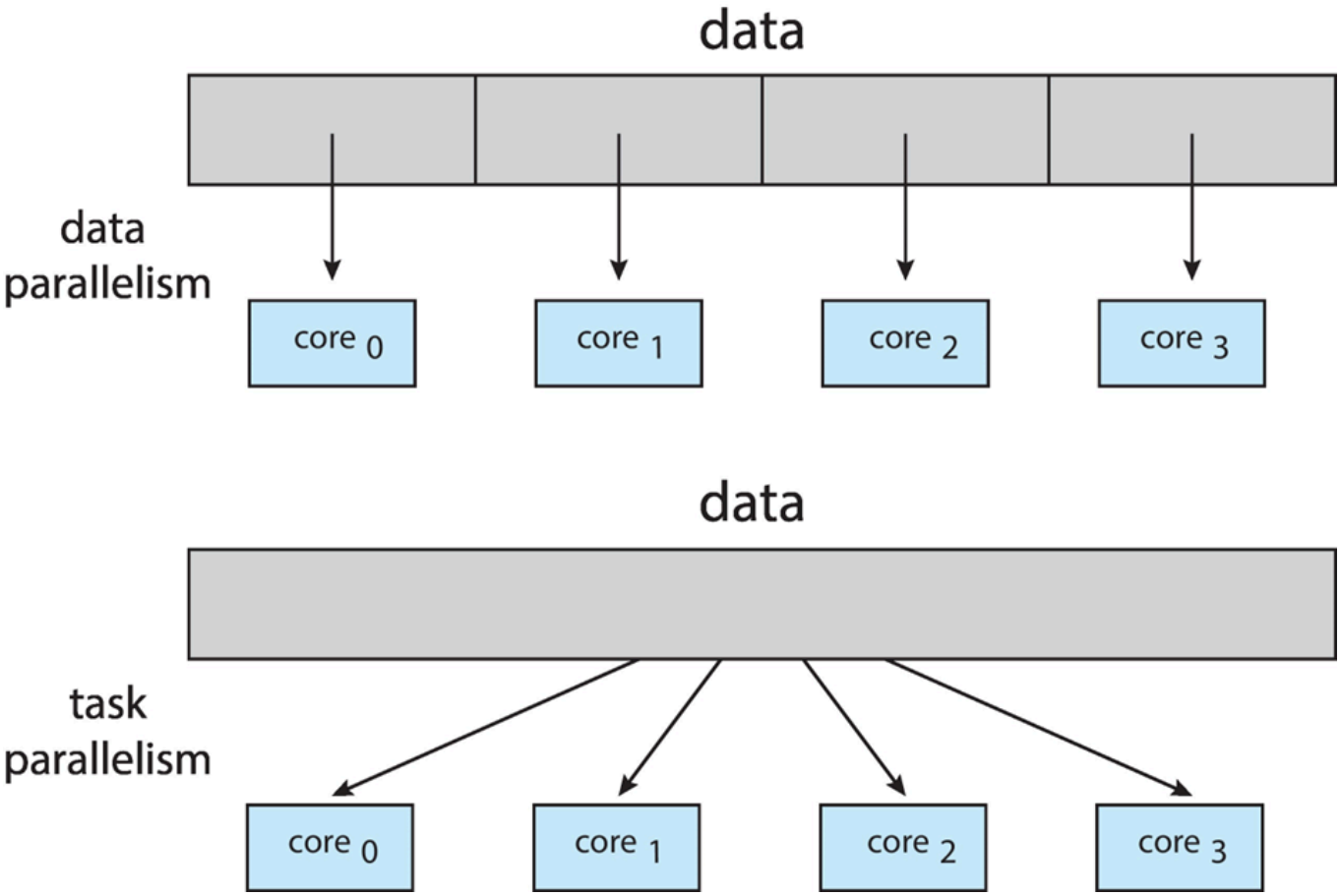


Parallelism:



Type of parallelism

- Data parallelism
  Distributes subsets of the same data across multiple cores and same operation on each
- Task parallelism
  Distributes threads across cores and each thread performing unique operation.



Amdahl's Law

The law identifies performance gains from adding additional cores to an application that has both serial and parallel components

$$Speedup \leq \frac{1}{S + \frac{1-S}{N}}$$

where S is serial portion and N is processing cores.

As N approaches infinity, speedup approaches $1/S$.

That is, serial portion of an application decides the upper limitation of speedup with multiple cores. The less serial portion is, the more benefits multiple cores have.

User Threads and Kernel Threads

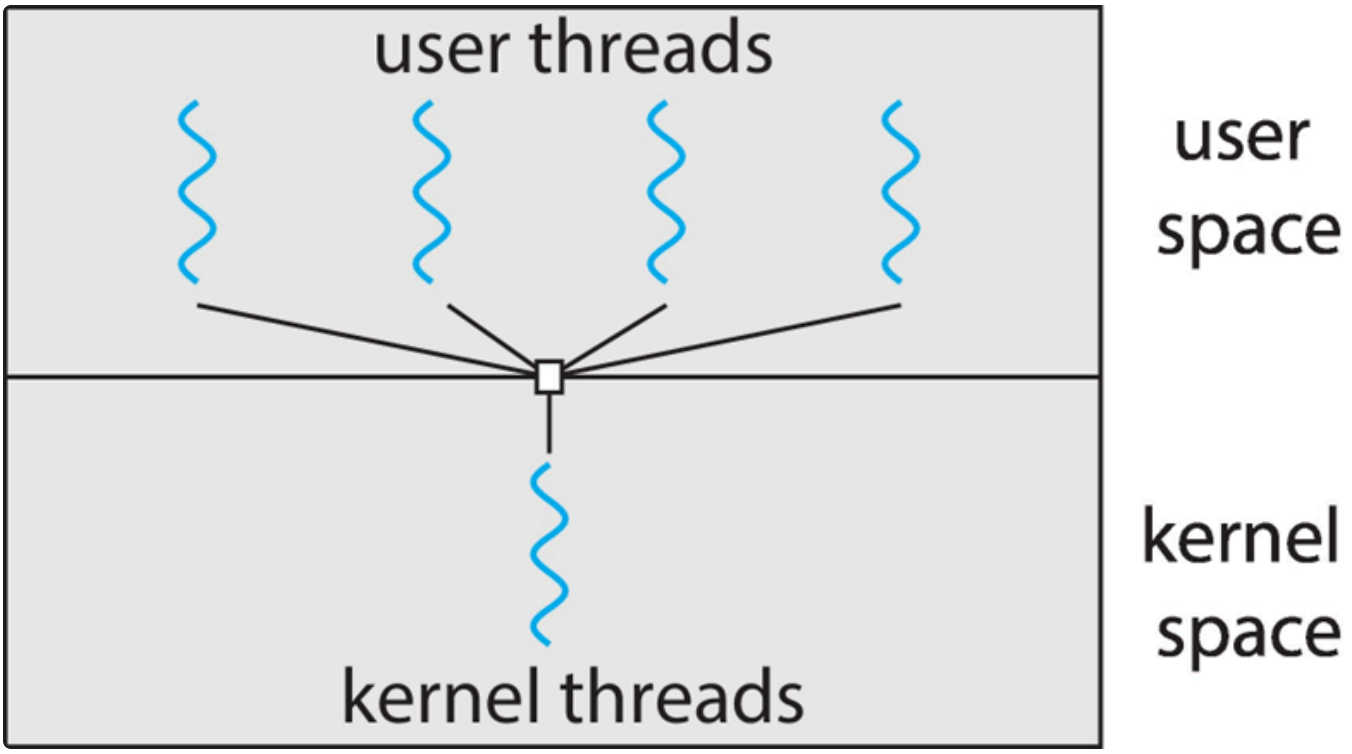
- User Threads
  Management done by user-level threads library. Three primary thread libraries are POSIX threads, Windows threads, Java threads
- Kernel threads
  Supported by Kernel. E.g., almost all general-purpose operating systems including Windows, Linux, etc.

Multithreading Models

- Many-to-One
- One-to-One
- Many-to-Many

Many-to-One

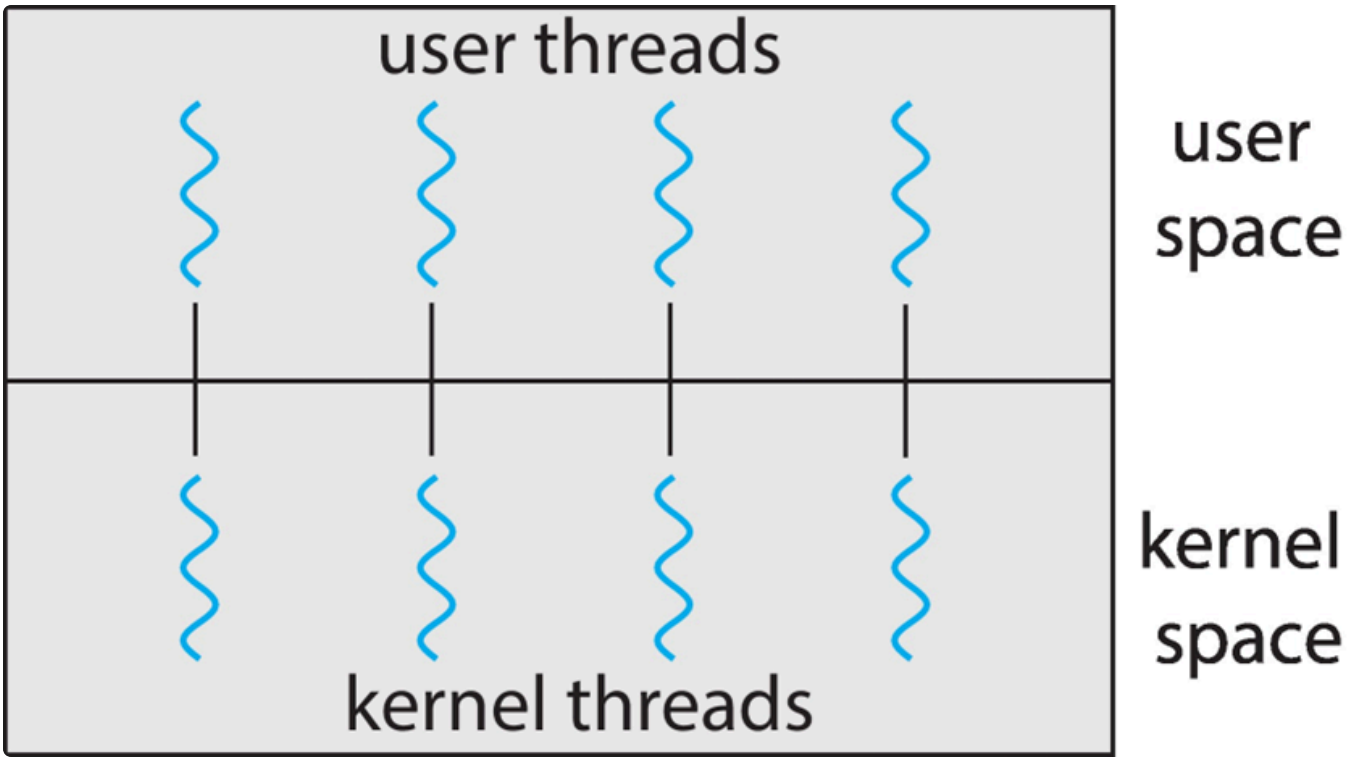
Many user-level threads mapped to single kernel thread.



- One thread blocking causes all to block
- Multiple threads may not run in parallel on multicore system since only one may be in kernel at a time
- Few systems currently use this model
- e.g., Solaris Green threads, GNU Portable threads

One-to-One

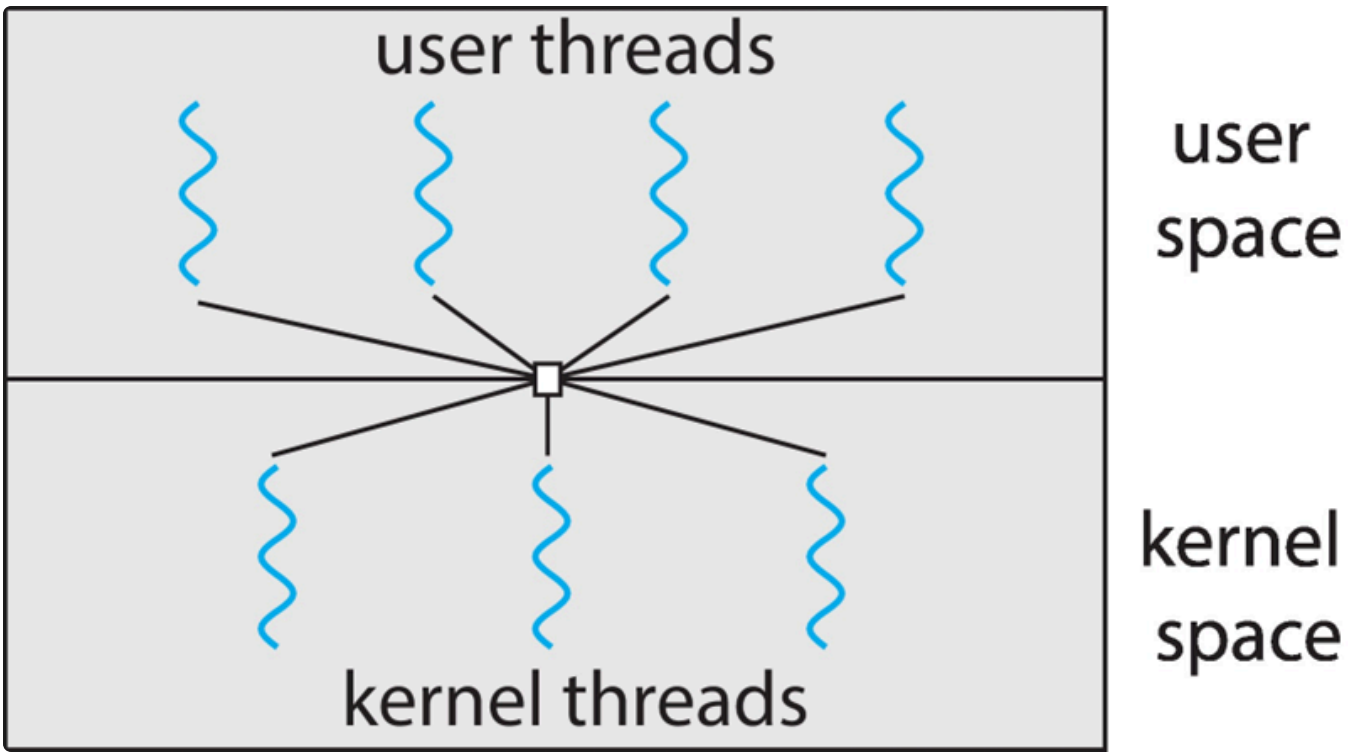
Each user-level thread maps to kernel thread.



- Creating a user-level thread creates a kernel thread
- More concurrency than many-to-one. (i.e., more efficient architecture)
- Number of threads per process sometimes restricted due to overhead.
- e.g.: Windows, Linux

Many-to-Many Model

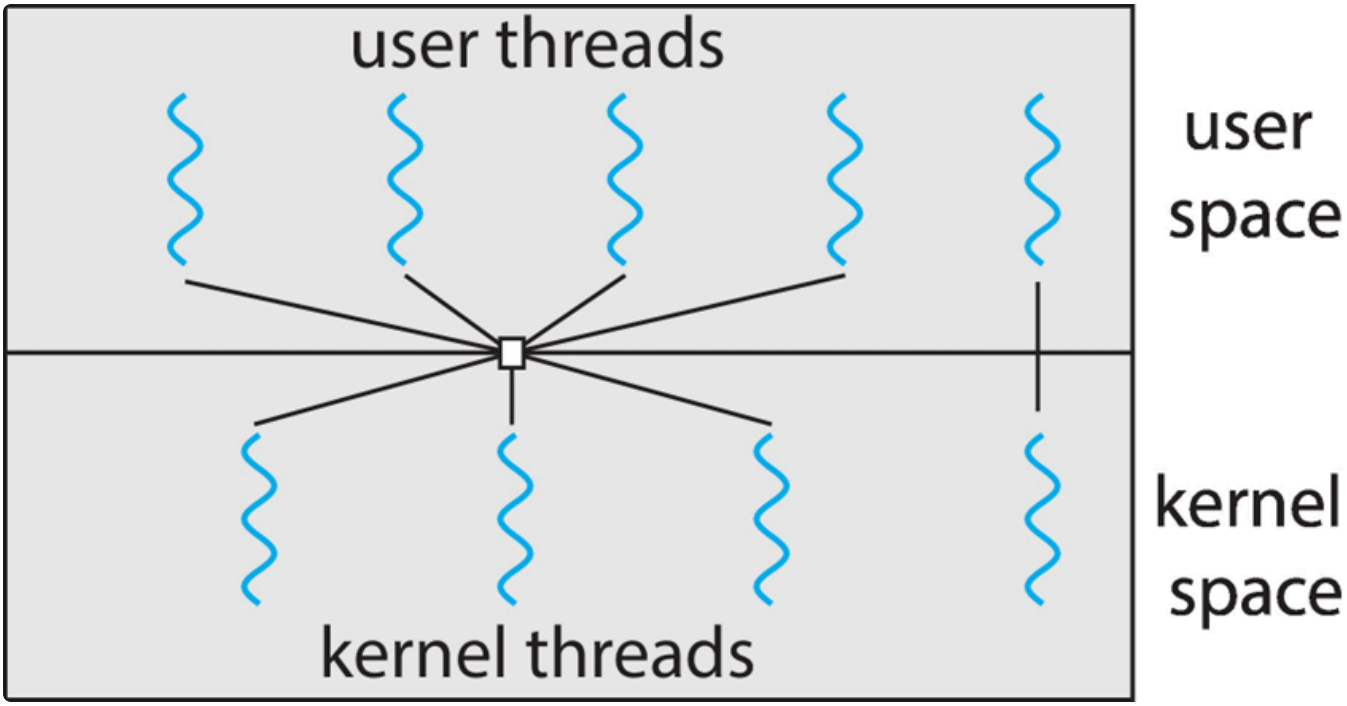
Allow many user level threads to be mapped to many kernel threads.



- Allows the operating system to create a sufficient number of kernel threads
- Not very common currently.
- E.g., Windows ThreadFiber

Two-level Model

Similar to M:M, except that it allows user thread to be bound to kernel thread.



Thread Libraries

Thread library provides programmer with API for creating and managing threads.
Two primary ways of implementing

- Library entirely in user space
- Kernel-level library supported by the OS

Pthreads

- May be provided either as user-level or kernel-level
- A POSIX standard API for thread creation and synchronization
- Specification, not implementation
- Common used in Unix/Linux

Windows Multithreaded C Program

Operations include creating threads, waiting for threads, etc.

Java Threads

- Managed by the JVM
- Implemented typically using the threads model provided by underlying OS
- Java threads may be created by
  - Extending thread class
  - Implementing the runnable interface(standard practice)
- Rather than explicitly creating threads, Java also allows thread creation around the Executor interface(implicitly).

Implicit Threading

Implicit threading grows in popularity as numbers of threads increase, program correctness more difficult with explicit threads.
For implicit threading, creation and management of threads done by compliers and run-time libraries rather than programmers.

Five methods:

- Thread Pools
- Fork-Join
- OpenMP
- Grand Central Dispatch
- Intel Threading building blocks

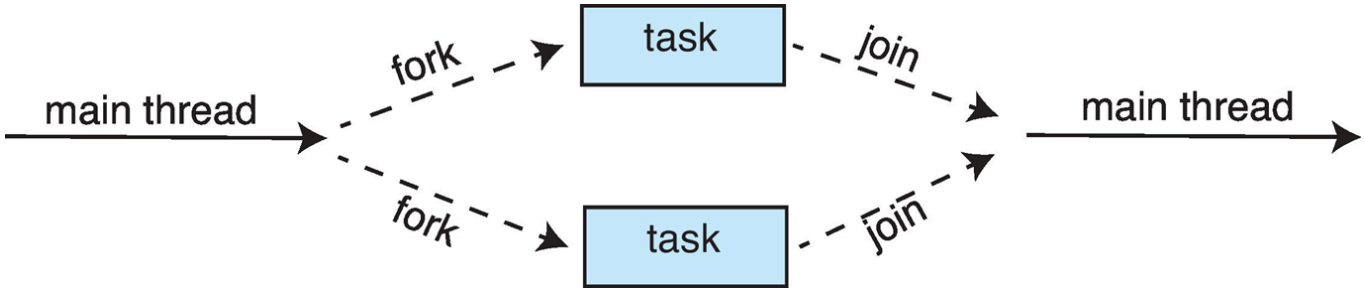
Thread Pools

Create a number of threads in a pool where they await work.
Advantages:

- Slightly faster to service a request with an existing thread than create a new thread
- Allows the number of threads in the applications to be bound to the size of the pool
- Separating task to be performed from mechanics of creating task allows different strategies for running task.

Fork-Join Parallelism

Multiple threads are forked and then joined.



Divide-and-Conquer. Fork -> Perform parallelly -> Join

OpenMP

It is through compiler to perform parallel regions of codes. Like `#pragma omp parallel`.

```
#pragma omp parallel for
for (i = 0; i < N; i++) {
   c[i] = a[i] + b[i];
}
```

Grand Central Dispatch

It is Apple technology for macOS and iOS operating systems. It allows identification of parallel sections and manages most of the details of threading.
Block is in `^{}`, like `^{printf("I am a block");}`. It is placed in dispatch queue and assigned to to available thread in thread pool when removed from queue.

Two types of dispatch queues:

- serial - blocks removed in FIFO and queue is serial called main queue.
- concurrent - removed in FIFO order but several may be removed at a time.

Intel Threading Building Blocks (TBB)

It is a template library for designing parallel C++ programs.

Threading Issues

- Semantics of `fork()` and `exec()` systems calls
- Signal handling: synchronous and asynchronous
- Thread cancellation of target thread: asynchronous or deferred
- Thread-local storage
- Scheduler Activations

Semantics of `fork()` and `exec()`

`fork()` may duplicate the calling threads or all threads. (Some UNIX have two versions of fork). `exec()` replaces the running process including all threads.

Signal Handling

Signals are used in UNIX systems to notify a process that a particular event has occurred. A signal handler is used to process signals.

- Generated by particular event
- Delivered to a process
- Handled by one of two signal handlers: default or user-defined.
  For multi-threaded, the signal may be delivered to certain thread, all thread, etc. to avoid intervening related threads.

Thread Cancellation

A thread is needed to be terminated before it has finished. Thread to be canceled is target thread.
Two general approaches:

- Asynchronous cancellation: terminate the target thread immediately
- Deferred cancellation: allows the target thread to periodically check if it should be cancelled.

Actual cancellation depends on the thread state. If thread has cancellation disabled, cancellation remains pending until thread enables it. Default type is deferred.

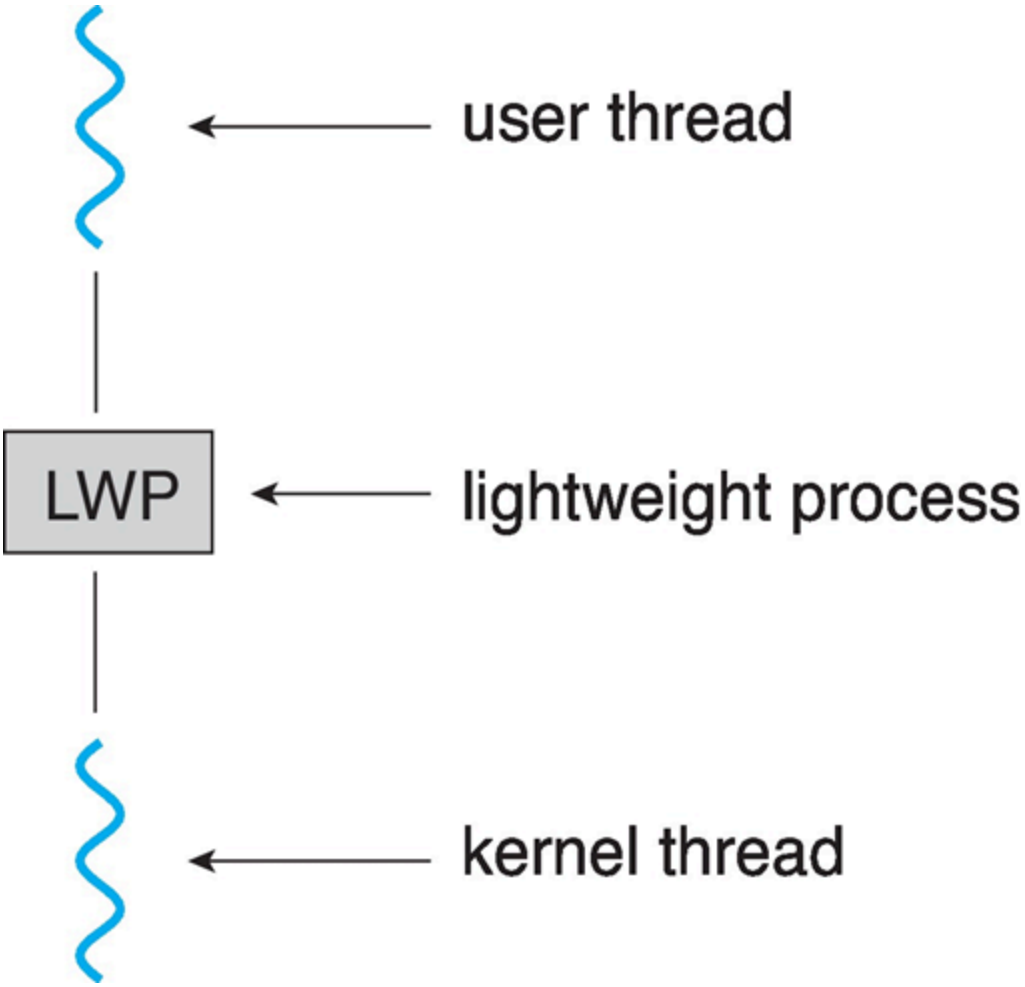
In Java, deferred cancellation uses `interrupt()` method, which sets the interrupted status of a thread. A thread can then check to see if it has been interrupted.

Thread-Local Storage

TLS allows each thread to have its own copy of data. It is visible across function invocations, different from local variables. Similar to static data but unique to each thread.

Scheduler Activations

M:M and two-level models require communication to maintain appropriate number of kernel threads allocated to application. It typically use an intermediate data structure between user and kernel threads - lightweight process(LWP).



Scheduler activations provide upcalls, a communication mechanism from the kernel to the upcall handler in the thread library. This communication allows an application to maintain the correct number kernel threads.

Operating System Examples

- Windows Threads
- Linux Threads

Windows Threads

- Implements the one-to-one mapping, kernel-level.
- Each threads contains:
  - A thread id
  - Register set representing state of processor
  - Separate user and kernel stacks for when thread runs in user mode or kernel mode.
  - Private data storage data area used by run-time libraries and dynamic link libraries(DLLs).
    Register set, stacks, and private storage area are known as the context of the thread.

Primary data structures of a thread include:

- ETHREAD(executive thread block) - includes pointer to process and to KTHREAD, in kernel space.
- KTHREAD(kernel thread block) - scheduling and synchronization info, kernel-mode stack, pointer to TEB, in kernel space.
- TEB(thread environment block) - thread id, user-mode stack, thread-local storage, in user space.

Linux Threads

Linux refers to them as tasks rather than threads.
Thread creation is done through `clone()` system call. It allows a child task to share the address space of the parent task.